

NLP

RNN LSTM y GRU

Docente:
Josselyn Ordóñez

Programa de la materia



Clase 1: Introducción a NLP, Vectorización de documentos.

Clase 2: Pre-procesamiento de texto. Word embeddings.

Clase 3: Modelos secuenciales: RNNs. Generación de secuencias.

Clase 4: Modelos secuenciales II: convolucionales y mecanismo de atención.

Clase 5: Modelos Seq2seq.

Clase 6: Transformers.

Clase 7: Grandes modelos de lenguaje. RAG.

Clase 8: Otros temas: Image captioning. ASR y TTS.

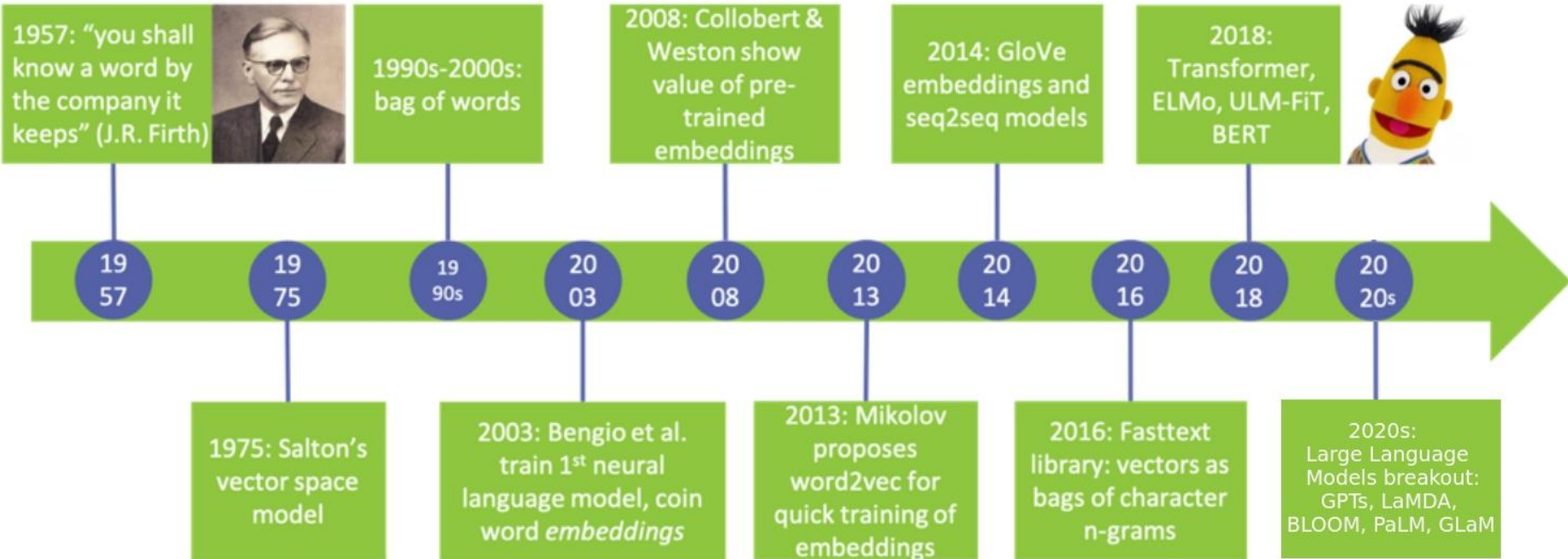
*Unidades con desafíos a presentar al finalizar el curso.

*Último desafío y cierre del contenido práctico del curso.

Timeline



1990: Celda RNN básica (Elman)
1997: LSTM



2016: LSTMs se convierten en SotA para traducción automática

Redes Neuronales Recurrentes (RNNs)



Es un tipo de neurona con un estado interno (o memoria) de manera que la información del pasado influye en los resultados futuros.



Se utiliza principalmente para resolver problemas de secuencia, en donde el valor anterior está relacionado con el valor futuro.



Permite construir modelos cuyos tamaños de secuencia sean potencialmente arbitrarios.

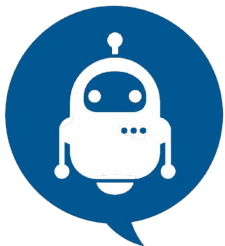


Implementa modelos de lenguaje de la forma:

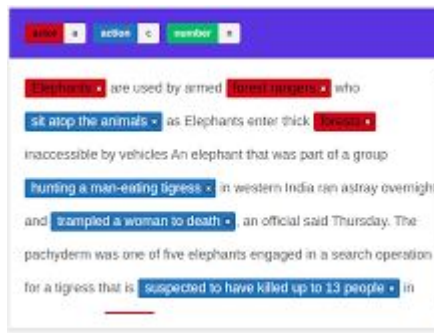
$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

*"Hoy el **día** está **hermoso** y **despejado**, se puede ver un hermoso **cielo... azul**"*

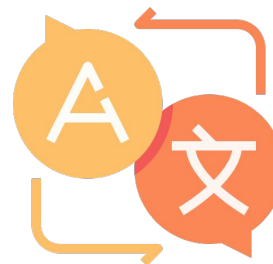
Algunos problemas de secuencia



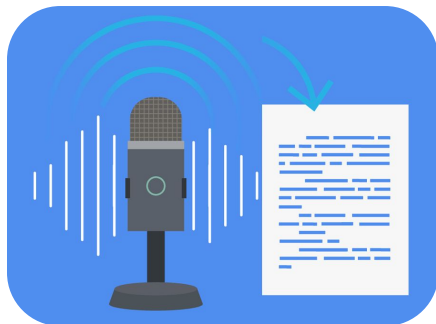
Bots
Conversacionales



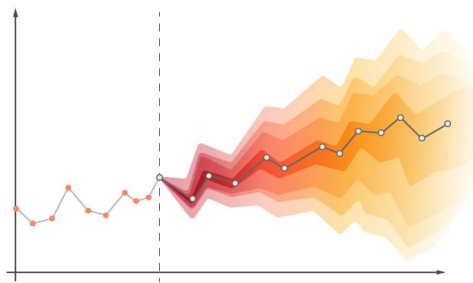
Name entity
recognition



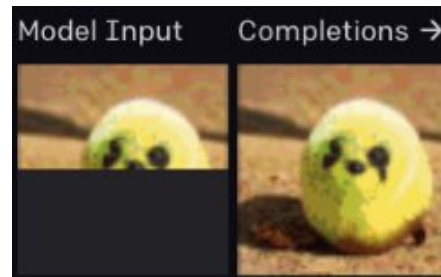
Traducción de
idiomas



Speech to text



Series
temporales



Completar una
imagen

Celda RNN básica (Elman)

[LINK](#)

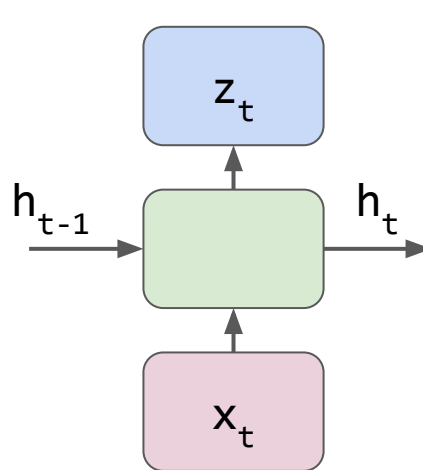
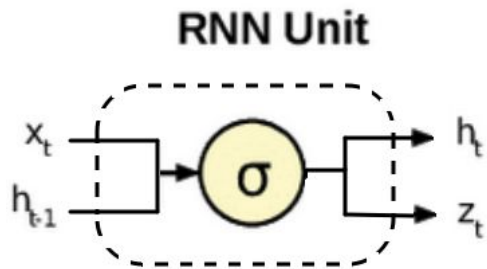
[API KERAS](#)



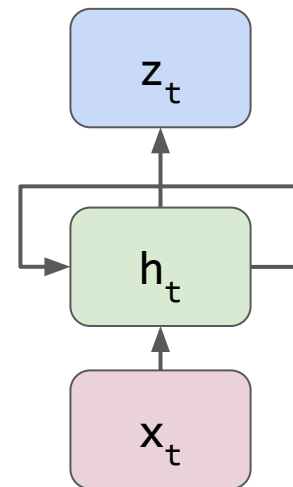
En Keras: SimpleRNN

$$h_t = \sigma(W_{hh} * h_{t-1} + W_{hx} * x + b_h)$$

$$z_t = h_t$$

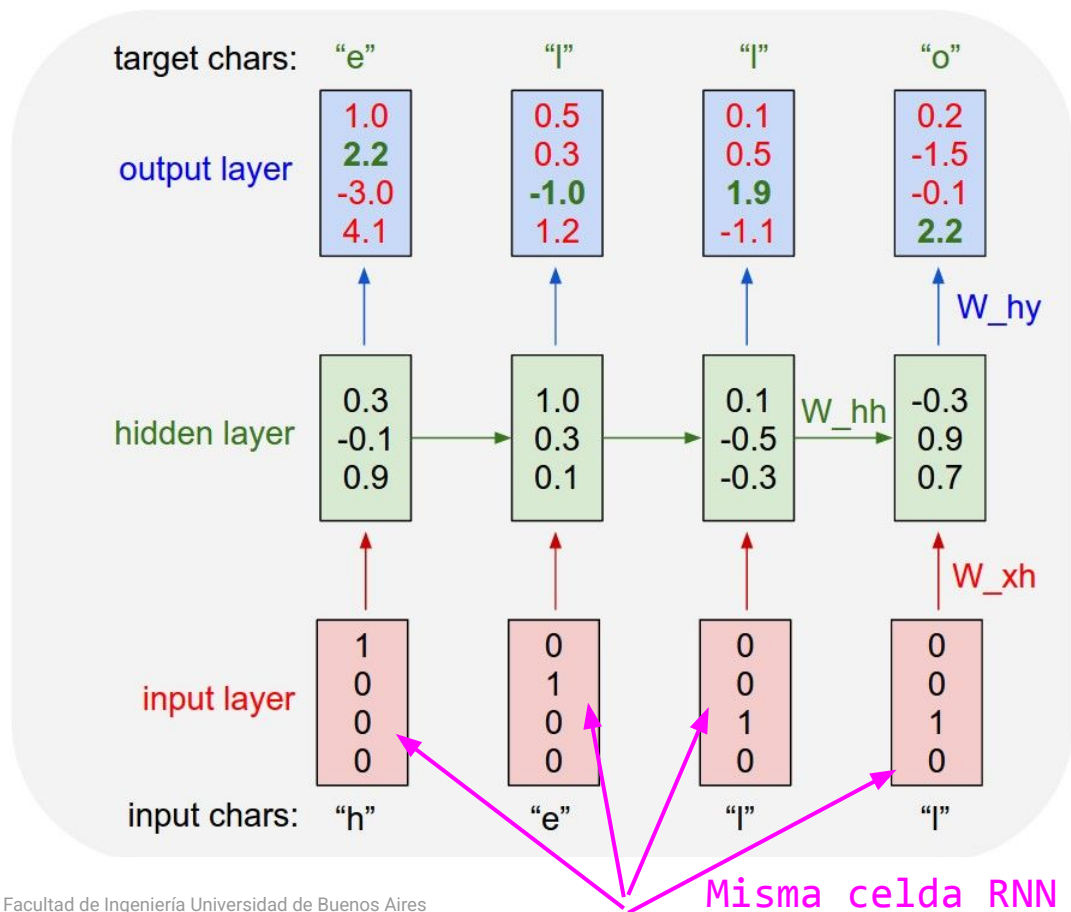


Unidad básica



Representación
compacta

Propagación (ejemplo) CAMBIAR DIAPO!!



En este ejemplo conceptual entra una palabra/letra y sale otra

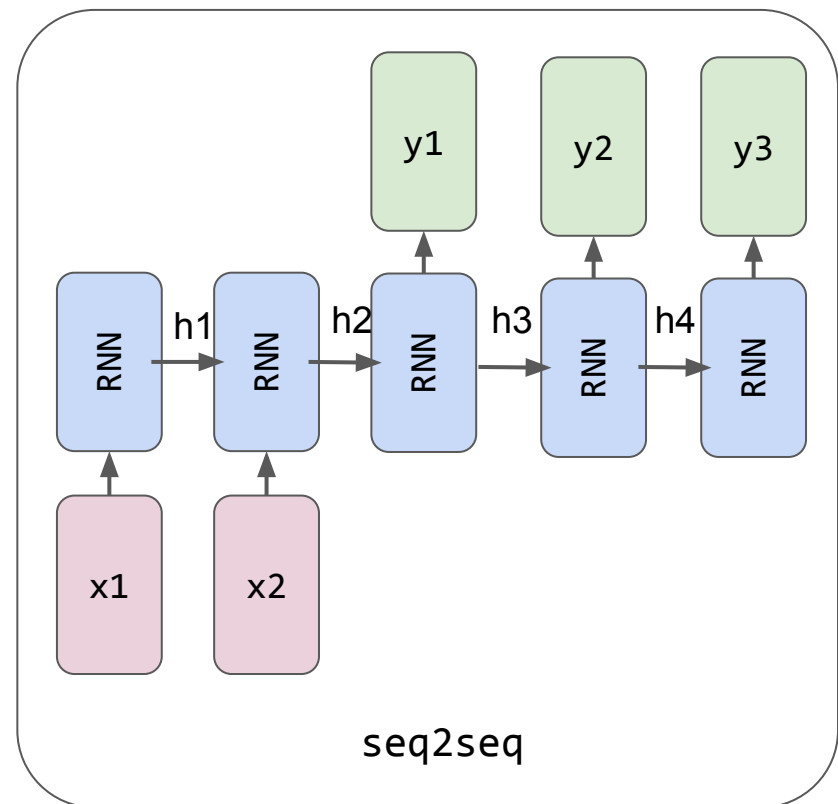
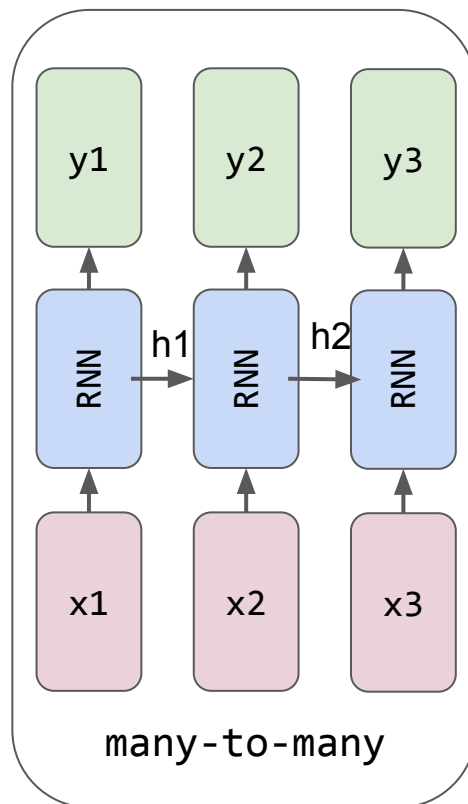
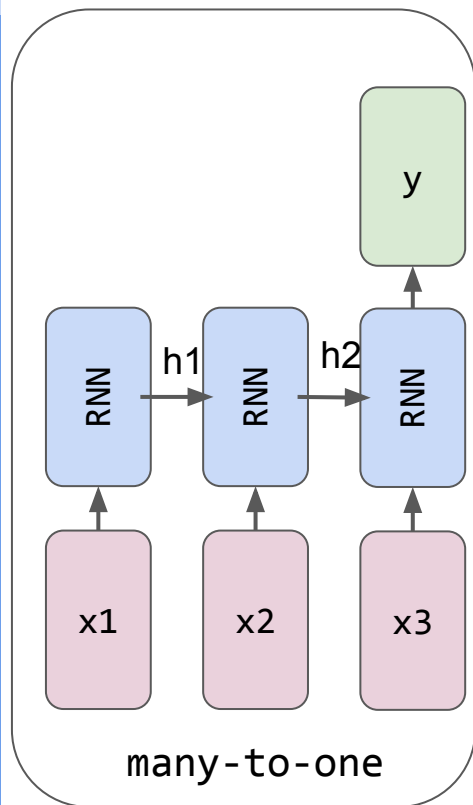
En estas redes de secuencia su grafo de cómputo es en serie, no es posible paralelizar, ya que el estado futuro depende del estado anterior.

Con cada salida se actualizan los pesos W_{hh} , W_{xh} y W_{hy} para el próximo cómputo

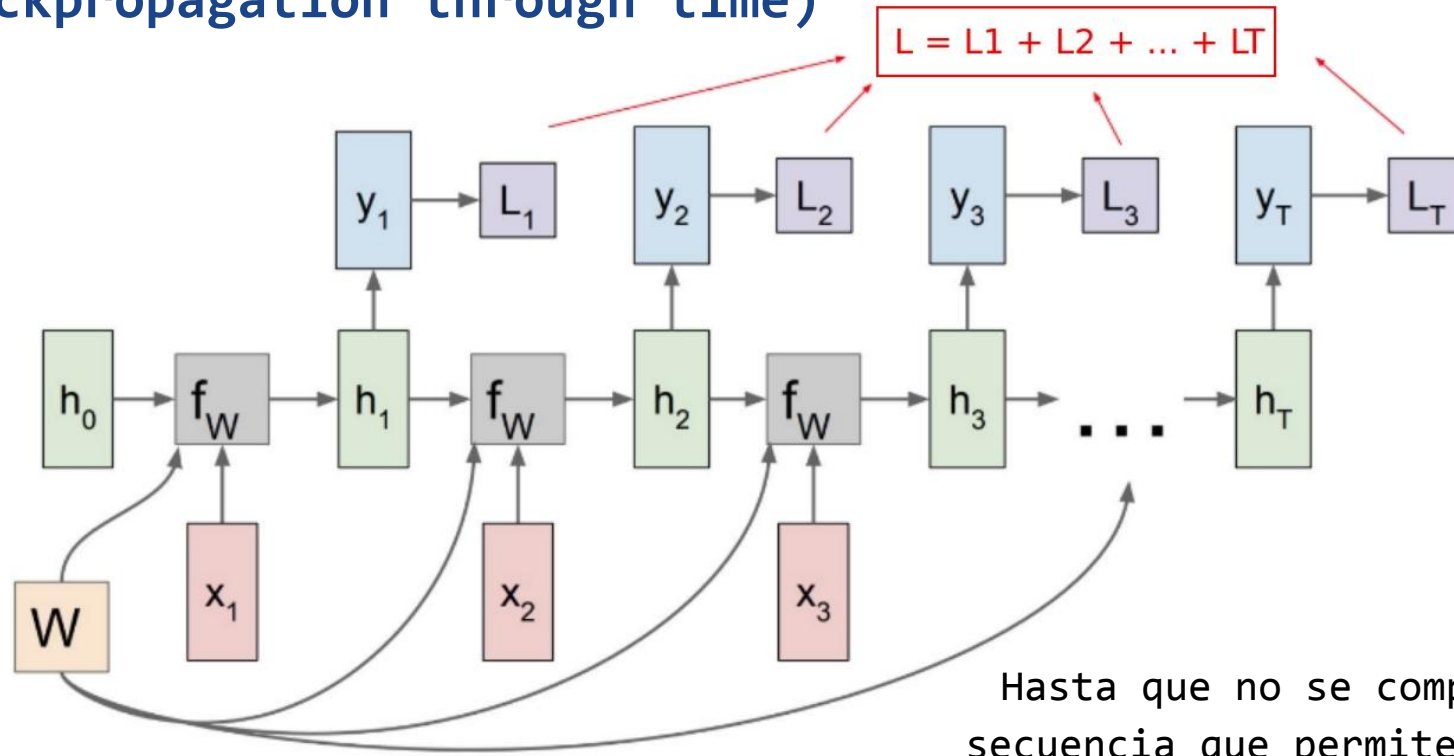
Misma celda RNN en diferentes instantes

Tipos de problemas de secuencia

[LINK](#)



Grafo de cómputo de una RNN y BPTT (Backpropagation through time)



Hasta que no se complete la
secuencia que permite calcular
el loss no se actualizan los
pesos (W) de la/s celda/s

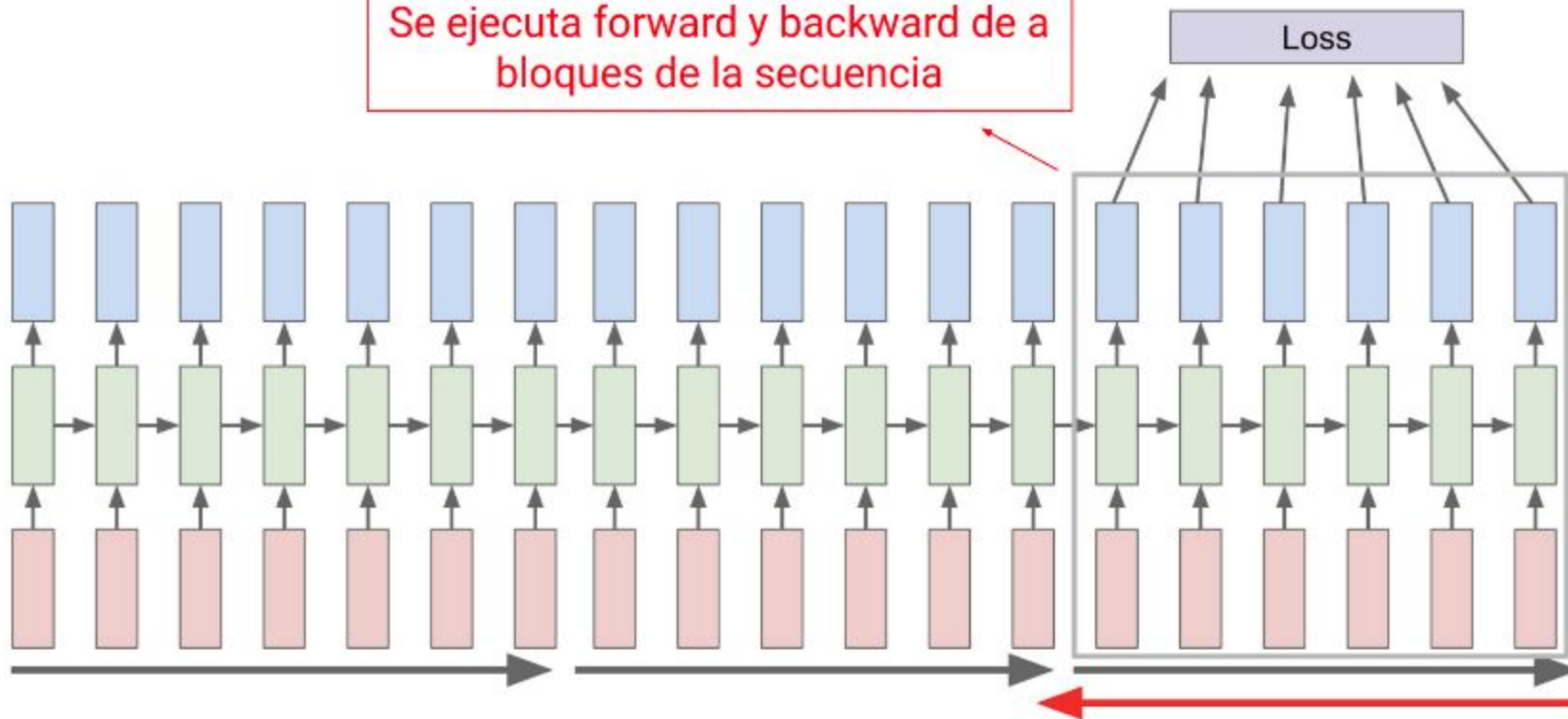
Forward & backward

CAMBIAR DIAPO!!

[LINK BACKPROPAGATION NUMPY](#)



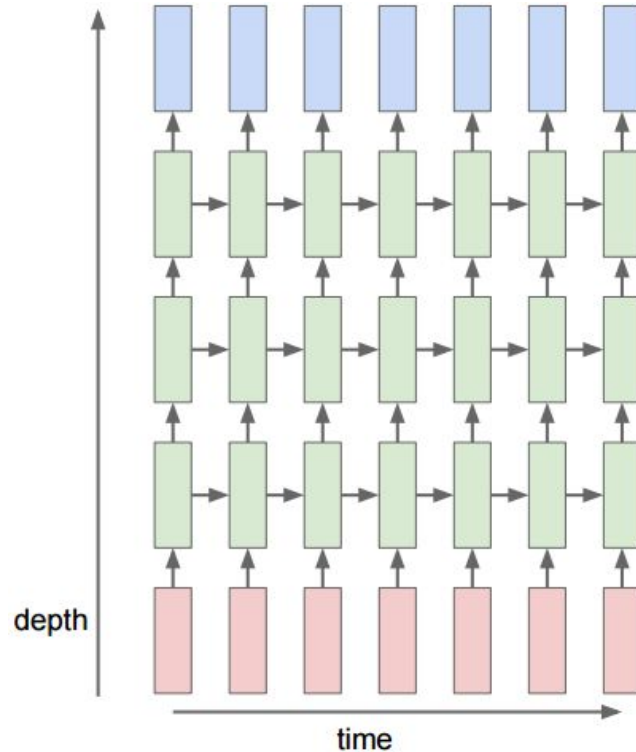
Se ejecuta forward y backward de bloques de la secuencia



Multi-layer RNN



Tal como se vio en los ejemplos se trata de apilar layers RNN en donde la salida de una se traslada a la entrada de la siguiente



Problema de una RNN tradicional

[LINK](#)



"Una RNN tradicional solo usa información del pasado y no de las futuras palabras para predecir"

Ejemplo: Data una sentencia determinar si existe una entidad que represente al nombre de una persona utilizando (name entity recognition)

Ejemplo 1:

*"Hoy escuche que **Victoria** terminó su bot para NLP" → persona*

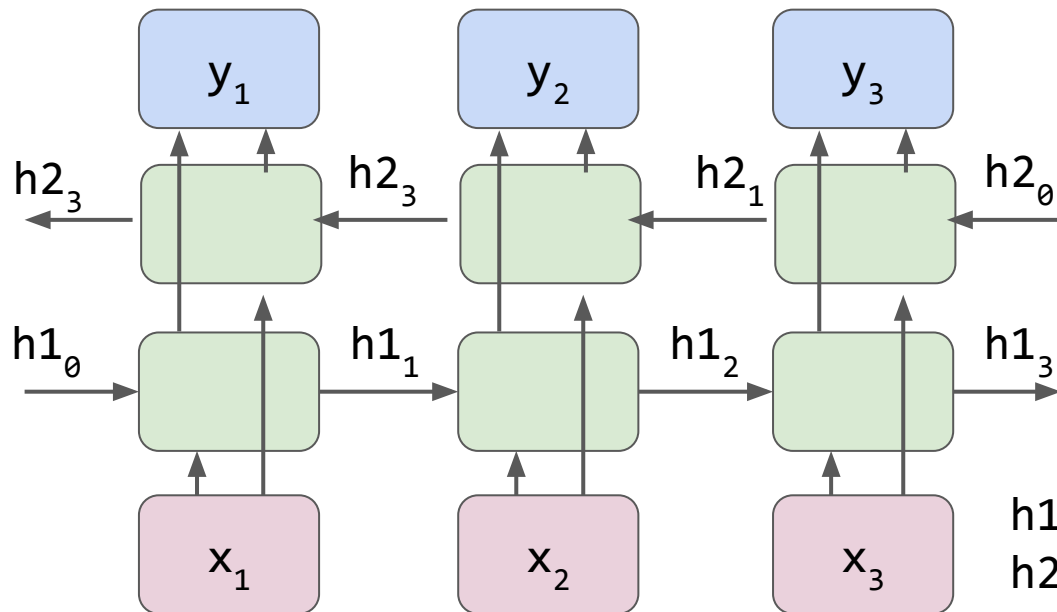
Ejemplo 2:

*"Hoy escuche que **Victoria** cambió de intendente" → ciudad*

La
contextualización
de la palabra es
a futuro



“La palabra anterior y la palabra futura tienen impacto en la presente predicción”

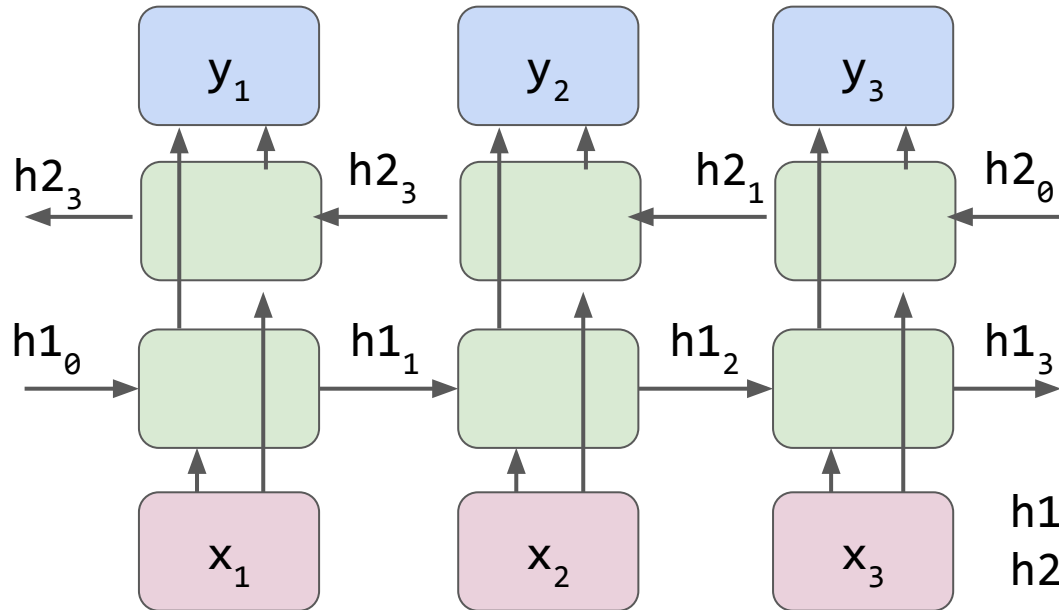


$$h1_t = \sigma(W_{hh1} * h1_{t-1} + W_{hx1} * x + b_1)$$
$$h2_t = \sigma(W_{hh2} * h2_{t+1} + W_{hx2} * x + b_2)$$

Las dos salidas pueden concatenarse, sumarse o promediarse



“La palabra anterior y la palabra futura tienen impacto en la presente predicción”



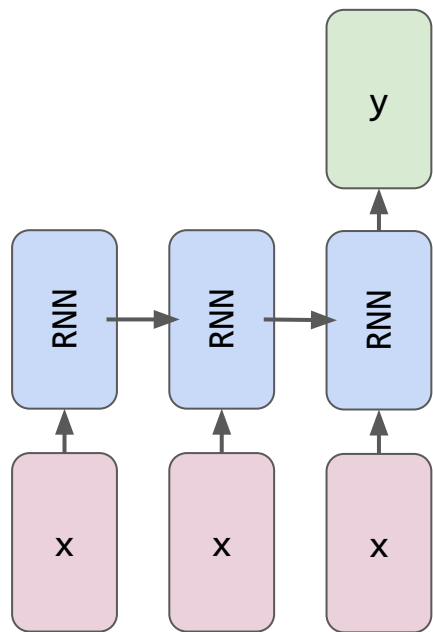
$$h1_t = \sigma(W_{hh1} * h1_{t-1} + W_{hx1} * x + b_1)$$
$$h2_t = \sigma(W_{hh2} * h2_{t+1} + W_{hx2} * x + b_2)$$

Las dos salidas pueden concatenarse, sumarse o promediarse

many-to-one



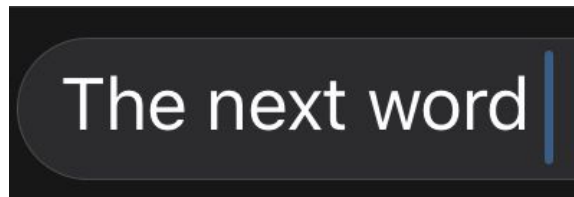
"Dada una sentencia o oración de entrada de tamaño fijo, el sistema arroja un único resultado que la representa".



many-to-one



Este tipo de estructuras se utilizan para determinar cuál es la siguiente palabra o elemento en la secuencia o para clasificación (sentiment analysis).



Predicción de
próxima palabra



Análisis de
sentimientos

Long short term memory (LSTM)

[LINK](#)



Se introduce este tipo de celda neuronal con mayor persistencia de memoria para lograr capturar relaciones de palabras a largo plazo.



Se crearon en 1997. Se adoptó como la layer principal para problemas de secuencia en 2014 hasta la aparición de los transformers en 2017.



Desplazaron completamente a las capas RNN simples (Elman), ya que el costo adicional de las LSTM es marginal respecto al beneficio que otorgan

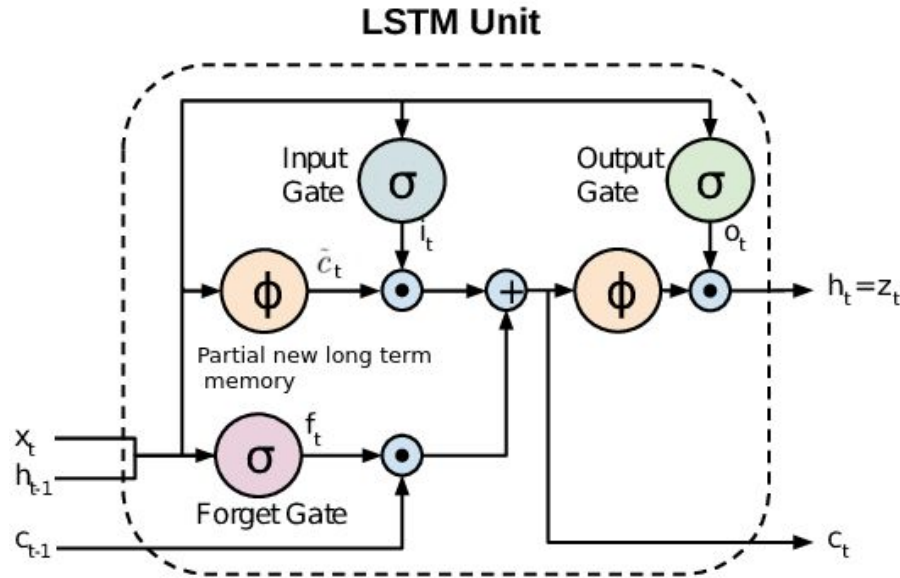


Se basan en el principio de ponderar la importancia de una palabra respecto al contexto futuro/pasado (key words).

¿Comprarías este producto?

*"**Incredible!** El producto es lo que venden, hace lo que tiene que hacer y me **ayudó mucho** a resolver los problemas que tenía. Lo **volvería a comprar** sin dudas"*

LSTM approach - Idea de la arquitectura



C: 'Memoria a largo plazo'

Input (i): Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget (f): Dada la entrada (X) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

Output (o): Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

$\cdot$: Producto elemento a elemento

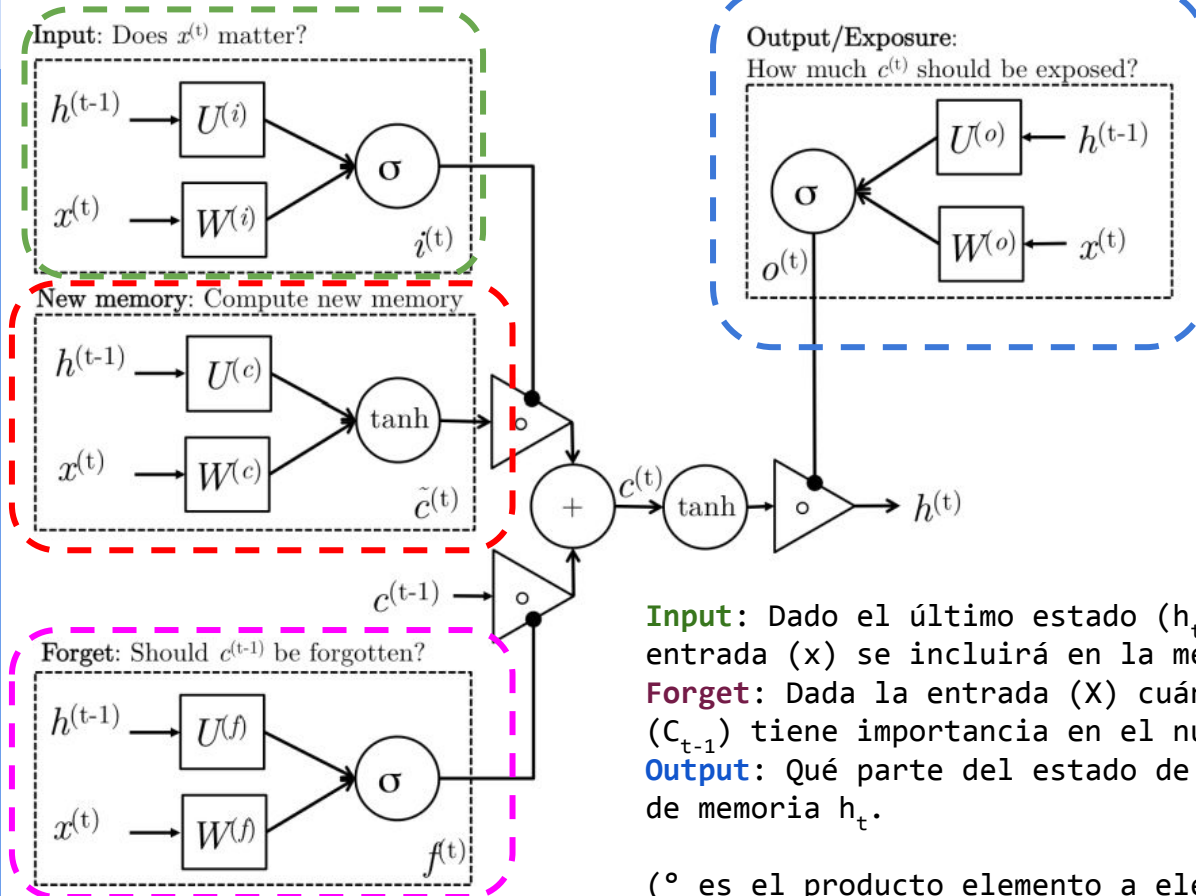
ϕ : Función Tanh

$+$: Suma vectorial

$\rightarrow$: Conexiones

LSTM approach

[LINK](#)



$$i_t = \sigma(W^{(i)}x_t + U^{(i)}h_{t-1})$$

$$f_t = \sigma(W^{(f)}x_t + U^{(f)}h_{t-1})$$

$$o_t = \sigma(W^{(o)}x_t + U^{(o)}h_{t-1})$$

$$\tilde{c}_t = \tanh(W^{(c)}x_t + U^{(c)}h_{t-1})$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \tanh(c_t)$$

Input: Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget: Dada la entrada (x) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

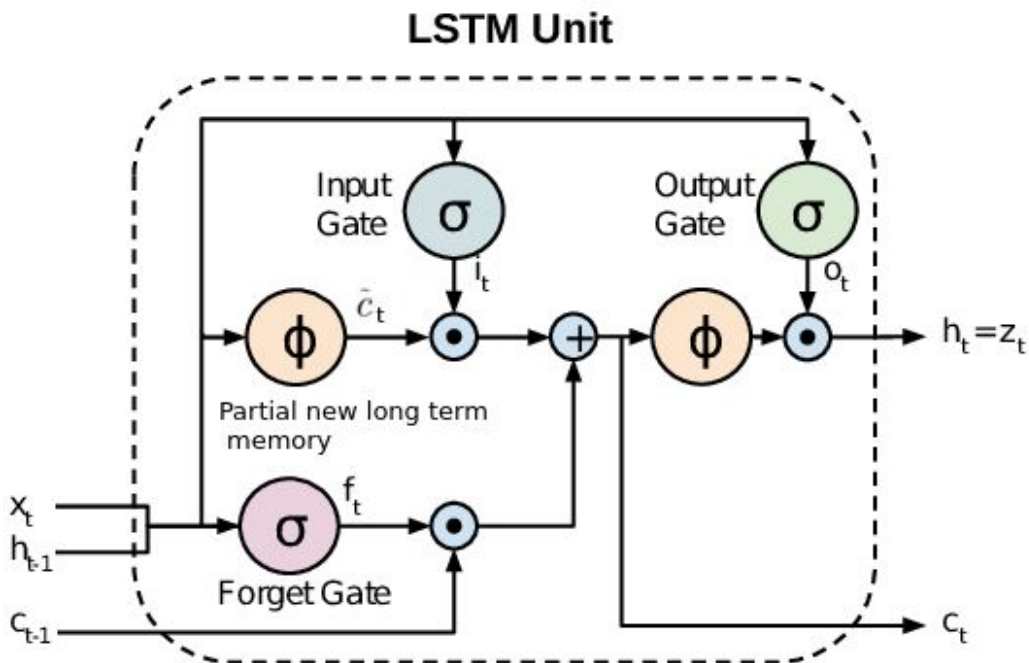
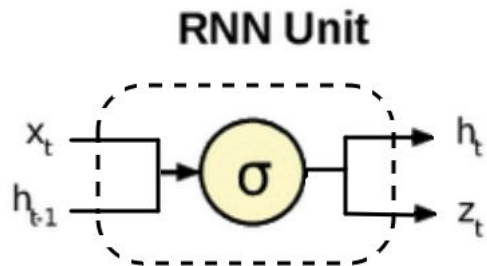
Output: Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

($\circ$ es el producto elemento a elemento)

LSTM vs RNN



La memoria de largo plazo c_t permite propagar gradiente eficientemente a mayor “profundidad temporal”.



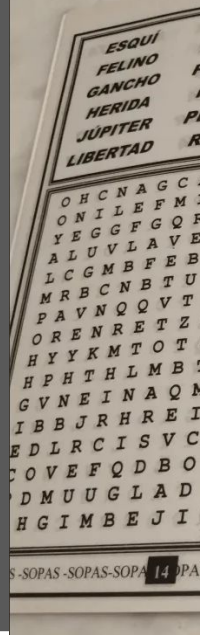
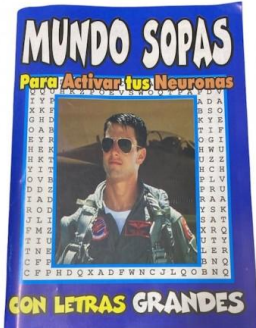
Ejemplo Many-to-one: Sopa de letras



Link al Colab



[LINK](#)

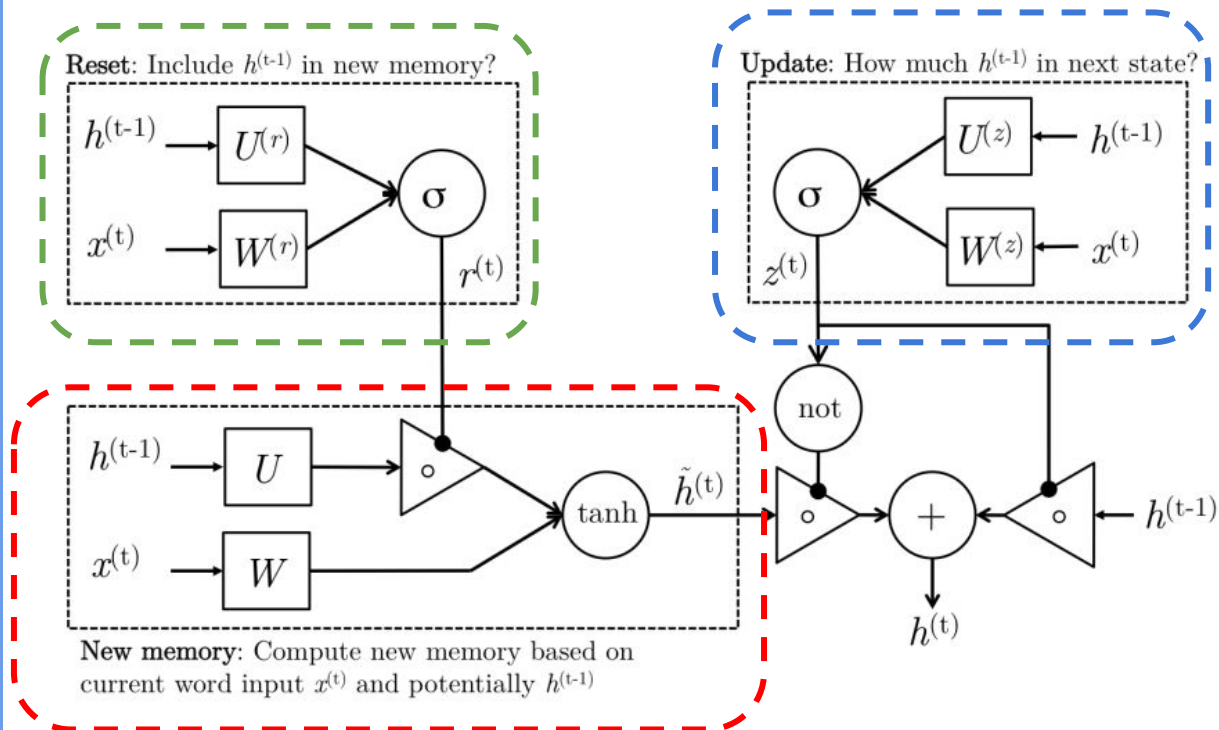


Gated Recurrent Units (GRU)

[LINK](#)



"Evolución de las RNN para superar problemas de "short-memory", versión reducida de una LSTM".



$$\begin{aligned} z_t &= \sigma(W^{(z)}x_t + U^{(z)}h_{t-1}) \\ r_t &= \sigma(W^{(r)}x_t + U^{(r)}h_{t-1}) \\ \tilde{h}_t &= \tanh(r_t \circ U h_{t-1} + W x_t) \\ h_t &= (1 - z_t) \circ \tilde{h}_t + z_t \circ h_{t-1} \end{aligned}$$

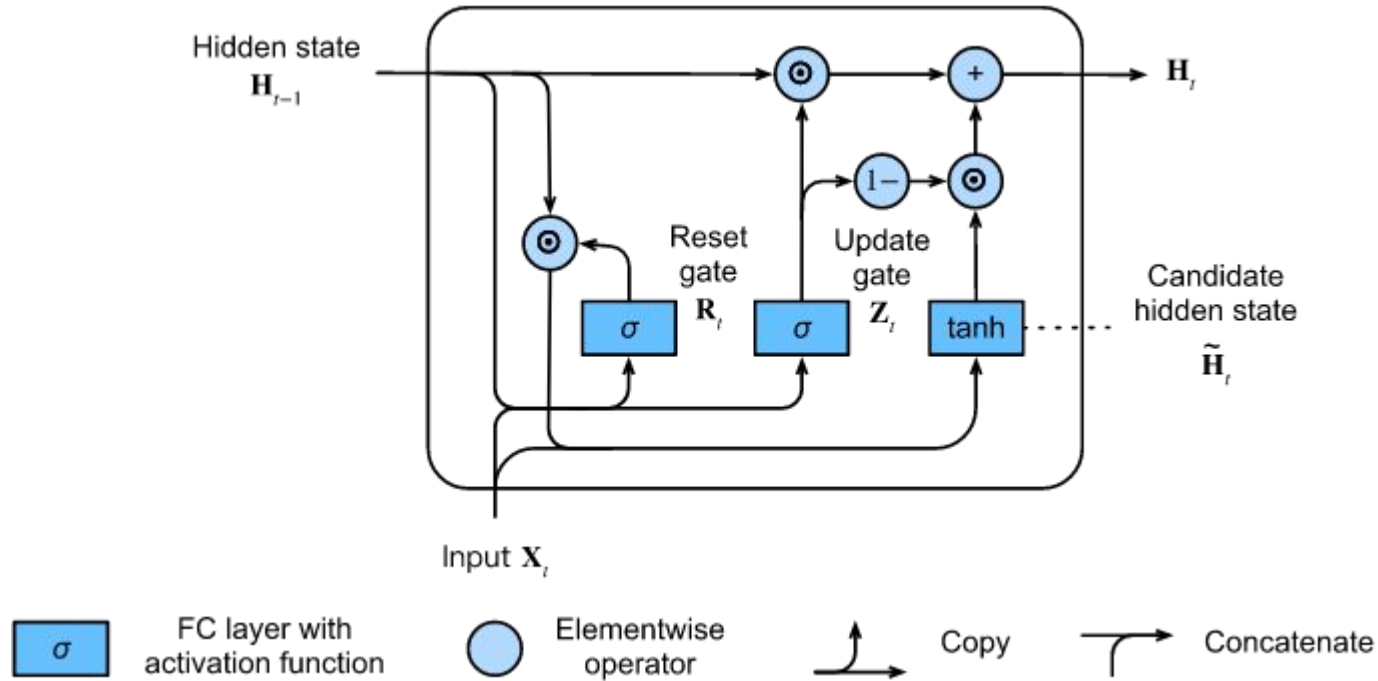
(Update gate)

(Reset gate)

(New memory)

(Hidden state)

Gated Recurrent Units (GRU)



Variantes de la LSTM



Peephole LSTM: Las compuertas forget, input y output acceden a la memoria de largo plazo. Disponible en los add-ons de TensorFlow.



Time-Aware LSTM: Permite representar la información de intervalos de tiempo transcurrido entre elementos de una secuencia.



Describir qué modelo de lenguaje vamos a entrenar

METER TOKENS DE FINALIZACIÓN E INICIO?

ORDENAR BIEN EL REPOSITORIO



Link al Colab



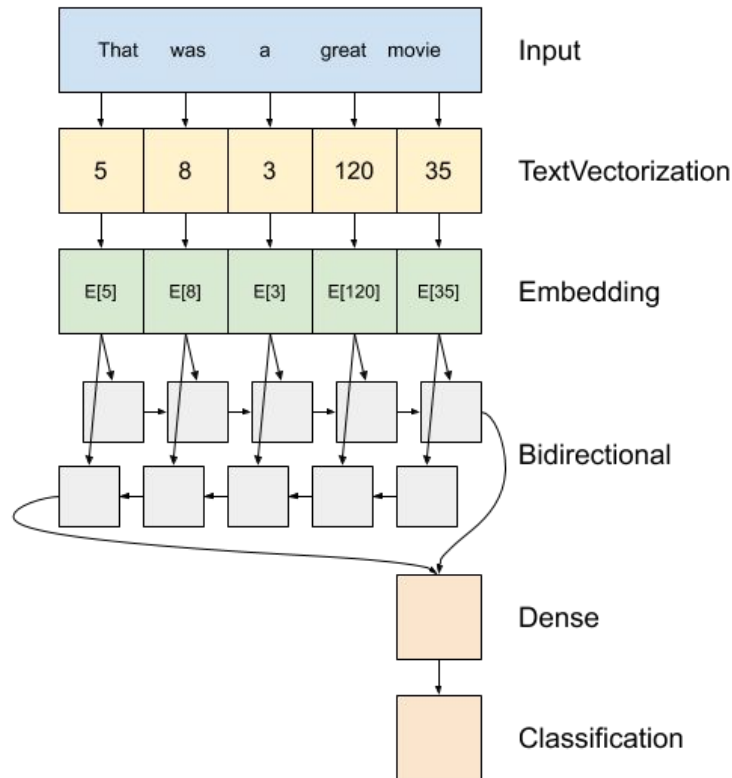
[LINK](#)

Embeddings + LSTM + Classifier

[LINK](#)



Arquitectura de alto nivel de un modelo “sentiment analysis”



Pre-trained Embedding layer

[LINK](#)

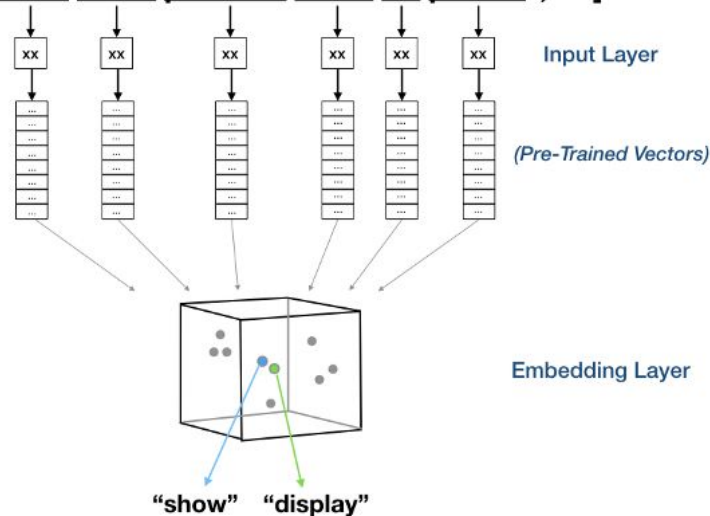
[LINK](#)



"Utilizar embeddings pre-entrenados (GloVe / FastText) en la layer de Embeddings de Keras"

```
Embedding(input_dim=vocab_size, # definido en el Tokenizador
          output_dim=embed_dim, # dimensión de los embeddings utilizados
          input_length=in_shape, # máxima sentencia de entrada
          weights=[embedding_matrix], # matrix de embeddings
          trainable=False)) # marcar como layer no entrenable
```

**[“I want to search for blood pressure result history”,
“Show blood pressure result for patient”, ...]**



a	1
am	2
as	3
act	4
all	5
...	6
...	7
...	8
...	9
...	10
...	11
...	12
...	...
...	100
...	...
...	1000
...	...
...	10000
...	...
...	...

Predicción de texto - Modelos de lenguaje



Objetivo: Entender la estructura estadística del lenguaje aprovechando su estructura secuencial.

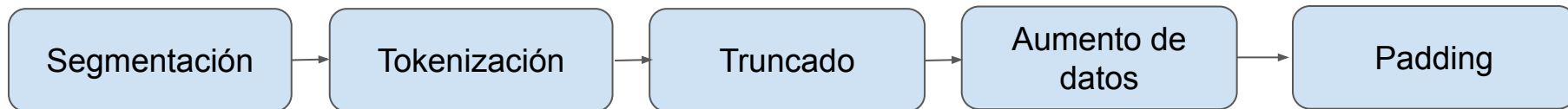
Tarea: Aprender a predecir la siguiente palabra.

$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

The next word |

$$(X_1, \dots, X_n) \longrightarrow X_{n+1}$$

Pasos a seguir:



Segmentación y Tokenización



Documentos:

`'Yesterday, all my troubles seemed so far away'`

Segmentación:

`['yesterday', 'all', 'my', 'troubles', 'seemed', 'so', 'far', 'away']`

Tokenización:

→ `[ 34, 189, 200, 1345, 8, 69, 12,753]`

Ventana de contexto



La **ventana de contexto** en un modelo de lenguaje es la cantidad máxima de tokens (palabras o subpalabras) que el modelo puede considerar a la vez como entrada. Define el alcance del "pasado" que el modelo puede usar para predecir la siguiente palabra o generar texto.

Si la secuencia es más **CORTA**
que la ventana de contexto



Padding (relleno)

Si la secuencia es más **LARGA**
que la ventana de contexto



Truncado

Truncado de secuencias:



Si tenemos una secuencia de tamaño M y una ventana de contexto de tamaño N tal que $M > N$, entonces podemos generar $1 + M - N$ secuencias de tamaño N :

Ejemplo ($M=6$, $N=3$)

["Todas", "las", "hojas", "son", "del", "viento"] se convierte en:

```
["Todas", "las", "hojas"]  
  ["las", "hojas", "son"]  
    ["hojas", "son", "del"]  
      ["son", "del", "viento"]
```

Data Augmentation y padding



Queremos que el modelo sea capaz de predecir cada elemento de la secuencia, no solo el último:

["Todas", "las", "hojas", "son", "del", "viento"] → [1, 2, 3, 4, 5, 6]

Ejemplo (M= 6, N=3)

["Todas", "las", "hojas"]

["las", "hojas", "son"]

["hojas", "son", "del"]

["son", "del", "viento"]

["Todas"] → [0, 0, 1]
["Todas", "las"] → [0, 1, 2]
["Todas", "las", "hojas"] → [1, 2, 3]

Probabilidad de una secuencia



$$\begin{aligned} P_{LM} (x^{T+1}, x^T, \dots, x^1) &= P_{LM} (x^{T+1} | x^T, \dots, x^1) P_{LM} (x^T, x^{T-1}, \dots, x^1) \\ &= P_{LM} (x^{T+1} | x^T, \dots, x^1) P_{LM} (x^T | x^{T-1}, \dots, x^1) P_{LM} (x^{T-1}, \dots, x^1) \\ &= \prod_{t=1}^T P_{LM} (x^{t+1} | x^t, \dots, x^1) \end{aligned}$$

Ejemplo:

$$P(\text{"La hojarasca crepitar"}) = P(\text{"crepitar"} | \text{"La hojarasca"}) P(\text{"hojarasca"} | \text{"la"}) P(\text{"la"})$$

Perplejidad: “Cantidad media de palabras posibles, en promedio.”



$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T}$$

Normalized by number of words

Inverse probability of corpus, according to Language Model

The

Guarda! Es el promedio geométrico en verdad: **16,49...**

[]

Midiendo desempeño en modelos de lenguaje: perplexity



$$\text{perplexity} = \underbrace{\prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)}_{\text{Inverse probability of corpus, according to Language Model}}^{1/T} \quad \leftarrow \text{Normalized by number of words}$$

Por cuestiones de estabilidad numérica conviene operar sobre los logaritmos de las probabilidades.

$$\text{PPL}(X) = \exp \left\{ -\frac{1}{t} \sum_i^t \log p_{\theta}(x_i | x_{<i}) \right\}$$

Modelo equiprobable: perplejidad V

Cross-entropy:

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

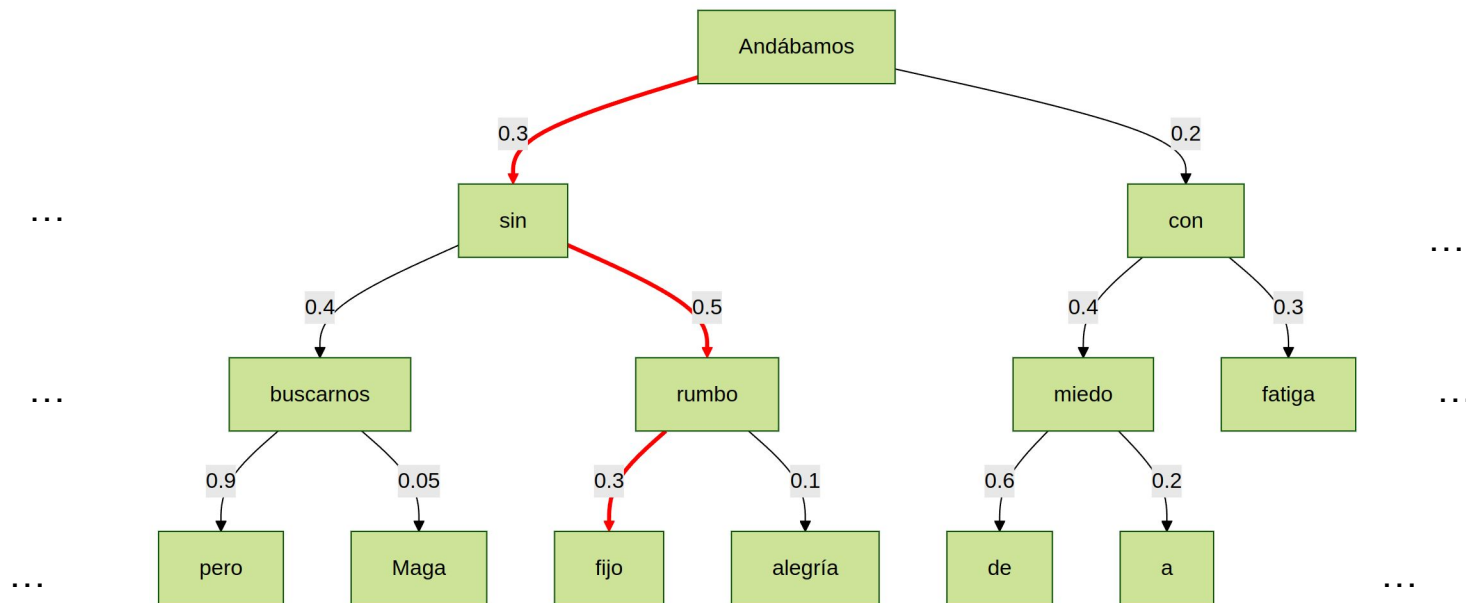


Link al Colab



LINK

Generación de texto en modelos de lenguaje: Greedy search

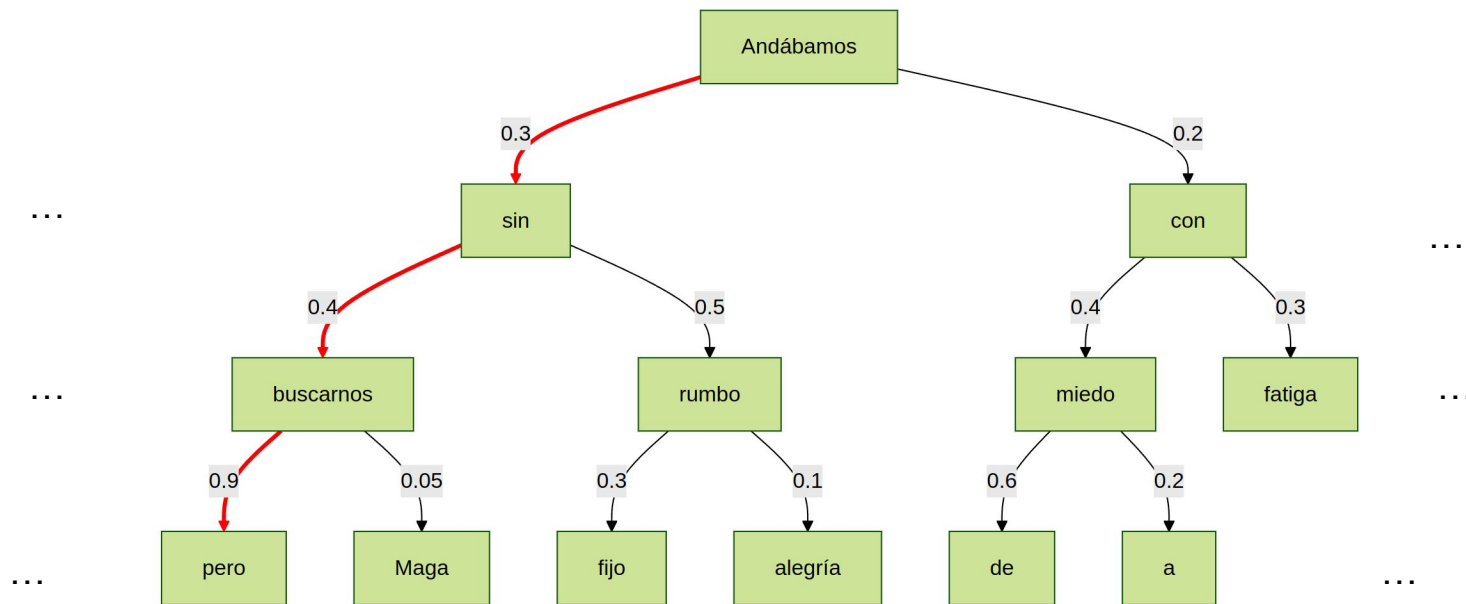


$$\prod_{t=1}^T P_{LM}(x^{t+1} | x^t, \dots, x^1) \longrightarrow$$

$P(\text{"Andábamos sin rumbo fijo"}) =$

$$0.3 \times 0.045 \times 0.3 = \mathbf{0.045}$$

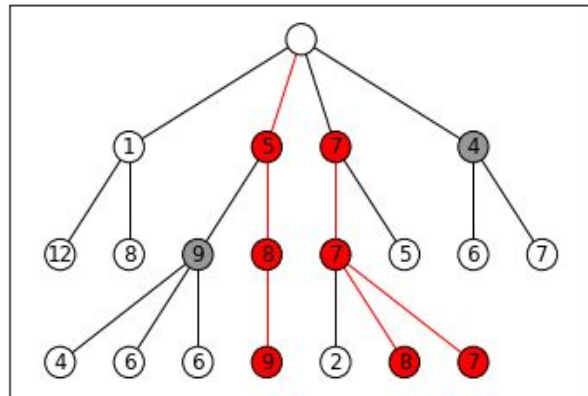
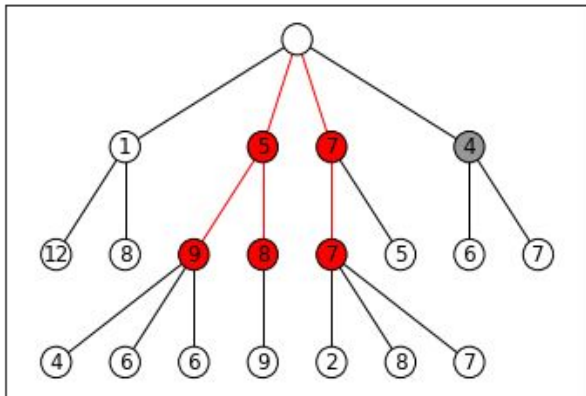
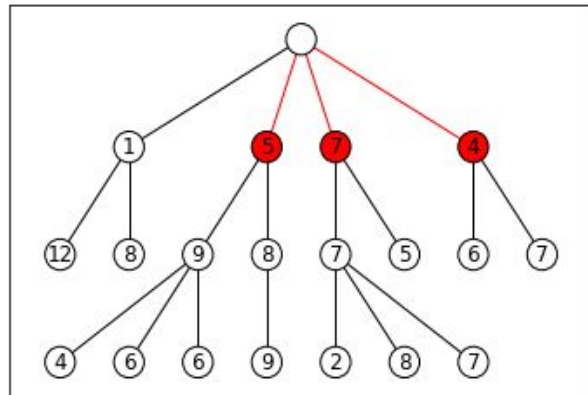
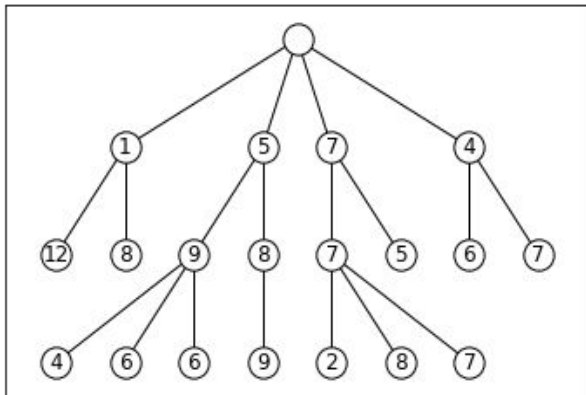
Generación de texto en modelos de lenguaje: Beam search



$$\prod_{t=1}^T P_{LM}(x^{t+1}|x^t, \dots, x^1) \longrightarrow$$

$$P(\text{"Andábamos sin buscarnos, pero"}) = \\ 0.3 \times 0.4 \times 0.9 = \mathbf{0.108}$$

Ejemplo: Beam search de ancho 3



Generación de texto en modelos de lenguaje: Muestreo aleatorio

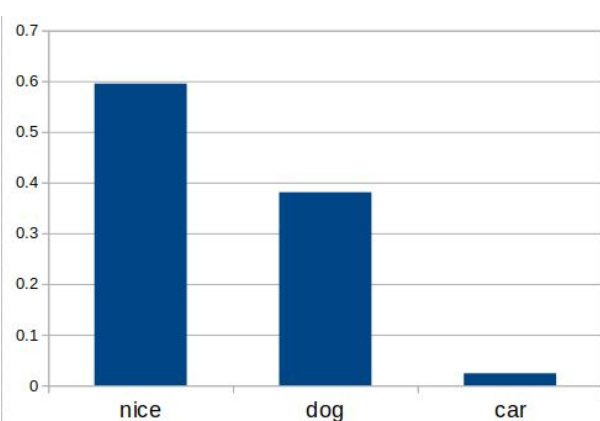


Ahora escogemos los
“beams” pesados por su
probabilidad.

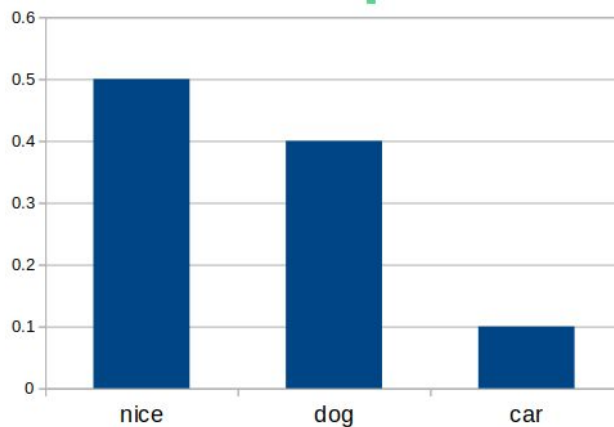
Generación de texto en modelos de lenguaje: Muestreo aleatorio con temperatura



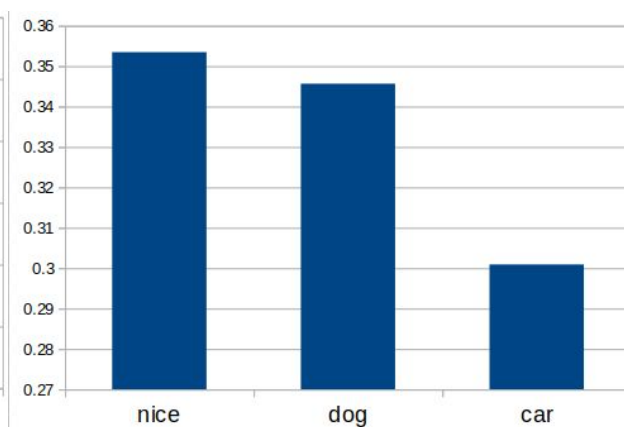
$$P_i = \frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^n e^{\frac{y_k}{T}}}$$



Temperatura = 0.5



Temperatura = 1



Temperatura = 10



Utilizar otro dataset y
poner en práctica
la generación de
secuencias con las
estrategias presentadas.



[LINK](#)



Top-K and Top-P (nucleus)

Top K: topear la cantidad de elementos entre las que estamos eligiendo

Top P: elegir suficientes términos hasta alcanzar una cierta probabilidad acumulada

